Software Tools and Systems Programming — Midterm

Midterm test

March 1st, 2023 – Duration: 2 hours

Total: 18 points

---

3 Problems

5 Questions

NAME	
Student Nbr	
Lecture Session	

Problems

Problem #1 [4 pts.]

Suppose that you have modularized your Assignment #1, so that the functions in charge of monitoring memory, CPU utilization and connected users are implemented in a file named "`monitor_utils.c`"; while the graphical representation, printing of information and any related functionality are implemented in "`output_utils.c`". Additionally you have included a library to support multi-colored schematics in the output, named "`ansi_colors`" which has to be used in the output module and executable in order to display information with colors. Assume that the header file of the library is located at `/usr/local/inc` while the library itself can be found at `/usr/local/lib`.

Create a makefile capable of compiling each of the modules, i.e. auxiliary files, main program, and generate the corresponding executable.

Include additional rules to clean intermediate sub-products such as object files, and to distribute the program in its current form.

Implement the makefile using: macros, variables, automatic variables, and special rules for self-describing documentation, i.e. including descriptive help for the rules.

Problem #2 [2 pts]

You are in charge of designing a critical component of a software module for the functioning of the James Webb Space Telescope (JWST). The software should continuously acquire data from the sensors in the Near Infra Red camera, but at the same time, it should periodically check for software updates which could come from different sources: NASA's headquarters, the International Space Station (ISS) or high-orbit geo-stationary satellites. Because in NASA and critical mission control operations, we believe in modular code, when an update is detected the application should initiate a new process in charge of receiving the updates and instantiate the update concurrently with the data acquisition process.

Create a pseudo-code representation of how this application will work, in particular focusing in the concurrent aspects and the logical sequence of the events.

Include an overall description and diagram of the strategy to be used and implemented.

Problem #3 [5 pts]

Write a C program that will search for a given genetic pattern in multiple strands of DNA. The program should receive multiple command line arguments, representing:

- the first command line argument, the genetic pattern to search for within the subsequent genetic sequences
- all the successive command line arguments, different genetic sequences to check for the presence of the genetic pattern given by the first command line argument.

The program should search **concurrently** through the multiple genetic sequences, returning by the end the total number of occurrences.

The program should also check that an appropriate number of command line arguments is passed, otherwise inform the user about it.

Any error messages should be directed to *standard error*.

You can assume that the genetic sequences are valid ones, following the same conventions and styles, such as all capital letters representing the four genetic bases.

```
./geneticSearch
This program requires at least two arguments:
   pattern seq1 [seq2 ...]
-----
./geneticSearch GTCAGT
This program requires at least two arguments:
   pattern seq1 [seq2 ...]
-----
./geneticSearch GTCA GTAGTCTTTGAGTCTTTG
Total matches: 0
-----
./geneticSearch GTCA GTAGTCTTTGAGTCTTTG GTCAGTCA
Total matches: 1
-----
./geneticSearch GTCA GTCAGTCA GTCATTTT GTCAGGGG
Total matches: 3
-----
./geneticSearch GTCA GTCAGTCA GTCATTTT GTCAGGGG GTCAGTCA GTCATTTT GTCAGGGG
Total matches: 6
-----
```

Questions

Please answer the following questions providing as much detail as possible:

Question #1 [1 pt]

(a) Describe the compilation process and the different stages that go on when generating an executable. (0.5 pts)

(b) Which flag(s) can be used to separate or differentiate these stages? (0.3 pts)

(c) Why would this be useful and how can we take advantage of it? (0.2 pts)

Question #2 [3 pt]

Consider the parts of the code given in the back of this page. Taking into consideration the memory address model seeing in the course, complete the following table including all the "variables" declarations in the code, specifying variable/memory structure, its location of the corresponding memory segment, size (or space) used, and when this memory space would be released.

```c
// includes
#include <stdio.h>
#include <string.h>
...
// end includes

char activation_Fn;
float Hubble_constant = 70;  // km/s/Mpc
float extra_dims = {3.141516, 2.71, 0.3333, 0.5, 1.0, 2.0};
int* univ_acc;
char* welcome_msg = "B09 mid-term test";

// functions declarations
void alpha_Fn() {
  int spin_star = 1;
  float mass_star;
  float stress_energy[9];
  char chemical_comps[] = {'H','N','P','S','K'};

  // ...
}

// main function
int main(int argc, char** argv) {
  char* restart_simulation;
  int* batch_nbr;
  float* local_time;
  char* msg = "Hello World!";
  int* xi;
  xi = malloc(sizeof(float));
  // ...
  alpha_Fn();
  // ...
  return 0;
}
```

variable declaration	**scope**	**Space (size)**	**Memory Address Region**	**de-allocation, when?**

Question #3 [1 pt]

(a) Define what are *Basic Input/Output Operations*. (0.35)

(b) Provide 5 examples of these operations and explain what each of these examples are doing. (0.1 per example+explanation)

(c) Why is important to close files when working with them? (0.15)

Question #4 [1 pt]

(a) Describe and explain what are and what is the meaning of "*file attributes*". (0.5)

(b) What could they be used for? (0.2)

(c) What shell command(s) can be used to control these? (0.3)

Question #5 [1 pt]

(a) What type of memory errors and/or problems may arise from improper manipulation, implementation and use of pointers? (0.5 pts)

(b) Provide explanations, examples and how the cases described in part a) can be fixed. (0.35 pts)

(c) What tools can be used to explore and investigate this type of problems? (0.15 pts)

Scratch/Extra space

Cheat Sheet

Make – Automatic variables

$@	The file name of the target of the rule.
$*	The file name of the target without the file extension.
$<	The name of the first prerequisite.
$?	The names of all the prerequisites that are newer than the target, with spaces between them.
$^	The names of all the prerequisites, with spaces between them. Discard duplicates.
$+	Similar to $^, but includes duplicates.

Some Useful C Functions

int execl(const char *path, const char *arg0, ... /*, (char *)0 */);	
int execvp(const char *file, char *argv[])	
int fclose(FILE *stream)	
char *fgets(char *s, int n, FILE *stream)	
pid_t fork(void)	
FILE *fopen(const char *file, const char *mode)	
int fprintf(FILE * restrict stream, const char * restrict format, ...);	
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);	
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);	
pid_t getpid(void);	
pid_t getppid(void);	
int kill(int pid, int signo)	
void *malloc(size_t size);	
int open(const char *path, int oflag)	
int scanf(const char *restrict format, ...);	scans input according to a format
char *strncat(char *dest, const char *src, size_t n)	concatenate strings
int strstr(char *strstr(const char s1, const char *s2);	locates the first occurrence of s2 in s1
int strcmp(const char *s1, const char *s2);	
int strncmp(const char *s1, const char *s2, size_t n)	compare strings
char *strncpy(char *dest, const char *src, size_t n)	copy strings
size_t strcspn(const char *s, const char *charset);	find offset of first character match
size_t strlen(const char *s)	string length
char *strpbrk(const char *s, const char *charset);	find characters in string
char *strrchr(const char *s, int c);	locate last occurrence of character in string
size_t strspn(const char *s, const char *charset);	find offset of first non-matching character
char *strtok(char *restrict str, const char *restrict sep);	string tokens
long strtol(const char *restrict str, char **restrict endptr, int base);	convert to long
int wait(int *status); int waitpid(int pid, int *stat, int options)	/* options = 0 or WNOHANG*/

Useful Macros

```
WIFEXITED(status) WEXITSTATUS(status) WIFSIGNALED(status) WTERMSIG(status) WIFSTOPPED(status)
WSTOPSIG(status)
```